Sessions WG update items

March 2026 MPI Forum
New York, NY

Some bookkeeping for March 2026 MPI Forum

- **Who's attending**
  - Sonja
  - Dominik

- **No need for WG time at the March meeting**
  - Tools/FT WG requests pertaining to Sessions?

- **https://github.com/mpiwg-sessions/sessions-issues/wiki**
  - Publications, docs, meetings summaries, …
  - Meeting time: Every 2nd Monday 08:00 AM Central Time
  - Lead: Howard Pritchard, (Dominik Huber)

Sessions WG features targeting MPI 6

- Process management related functionality:

  1. **Sessions friendly spawn mechanism** (plenary by Sonja Happ)

  2. **Updated publish/lookup API**

  3. **Adoption of dynamic process set mechanisms**
     - for supporting dynamic resources/elasticity/malleability
     - (has overlap with ULFM?). At least avoid conflicts.

  4. **Use side document mechanism**
     - for specification of some concepts, for example the COL being considered for dynamic process sets

Sessions WG features targeting MPI 6

- Additional items

  5. Sessions attributes (assists with language bindings implementations)

  6. Accessor methods for getting session handles from other MPI objects (groups, comms, files, windows) to assist tools.

1. Sessions friendly Spawn Mechanism

- **Why?** MPI_Comm_spawn does not work with Sessions
- **Approach:** Decouple spawn from communicators

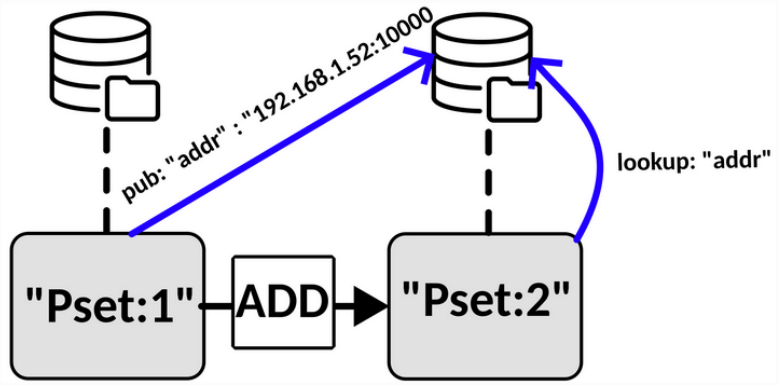
- => Plenary “Blocking and Non-blocking Spawn for Sessions“
  - Sonja Happ

2. Updated publish/lookup API

- **Why?** No MPI_COMM_WORLD + dynamic process creation
  => comm-independent information exchange required

- **MPI already has a (very minimal) functionality like this**
  - MPI_publish_name, MPI_lookup_name (port info for accept/connect)

- **Make it more generic: (named, global dictionaries)**
  - MPI_Dict_create(dict_name, info)
  - MPI_Dict_remove(dict_name)
  - MPI_Dict_data_publish(dictname, keys_values, info)
  - MPI_Dict_data_lookup(dictname, keys_values, info)
  - MPI_Dict_data_unpublish(dictname, keys)



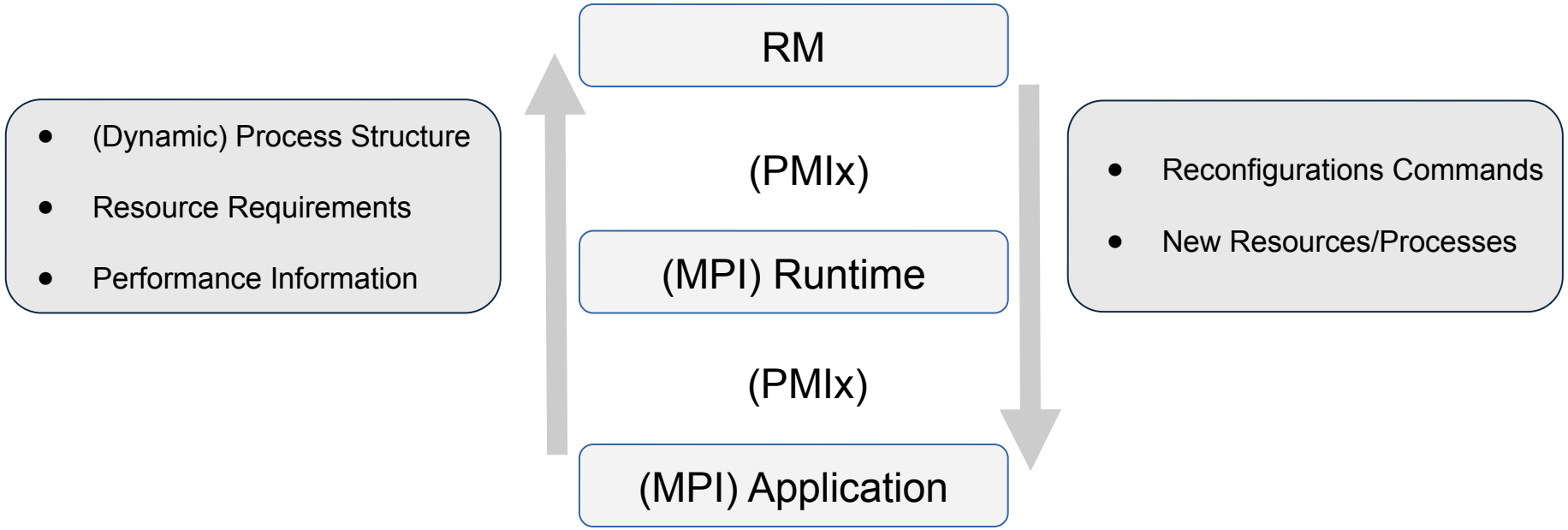
3. Dynamic PSets mechanism:

- **Why? Enable Dynamic Resource Management in HPC**
  - Dynamically change resources (and number of procs) assigned to (MPI) applications at runtime
- **Benefits:**
  - **Application Perspective:**
    - Tailor resources to (dynamically changing) resource requirements (e.g. AMR, pilot jobs, in-situ, ...)
    - Possibly enhance proactive fault tolerance mechanisms (e.g. migrate if resource is likely to fail)
  - **System Perspective:**
    - Improves scheduling flexibility & system metrics (throughput, energy-efficiency)
    - Address urgent/on-demand computing requirements
    - Enables reacting to system changes (node failures, congestion, etc.)

3. Dynamic PSets mechanism

- **Dynamic Processes in the MPI World model:**
  - **Removing Processes:**
    - MPI_Comm_disconnect (NOT possible on MPI_COMM_WORLD)
    - MPI_Finalize (is collective over MPI_COMM_WORLD)
    - MPI_Abort (most implementations abort all processes in the job)
    - No integration with RM
  - **Adding Processes:**
    - MPI_Comm_spawn/accept/connect (blocking)
    - no integration with RM

3. Dynamic PSets mechanism



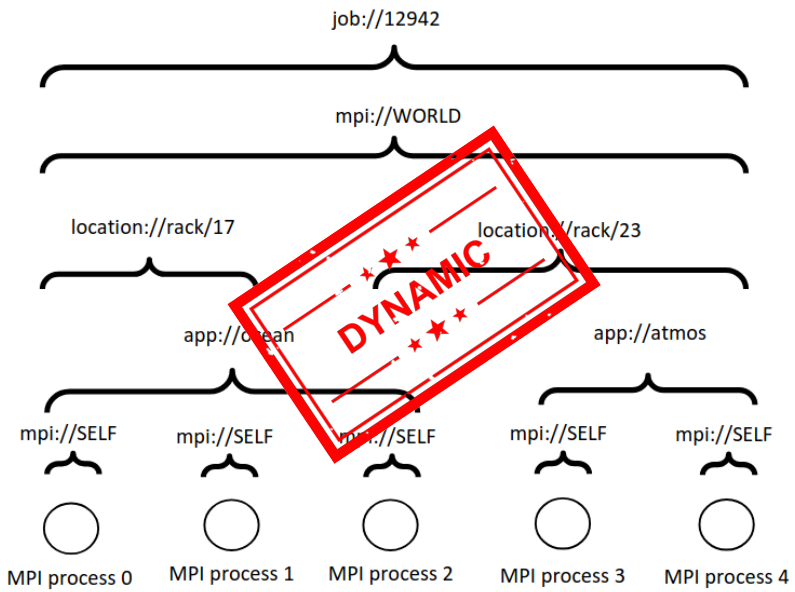
3. Dynamic PSets mechanism

Basis: MPI Sessions & MPI Process Sets

- No MPI_COMM_WORLD!
- PSets: Unique reference to set of processes
- Allows fine-grained creation of MPI groups & comms

(Co-)Design Goals:

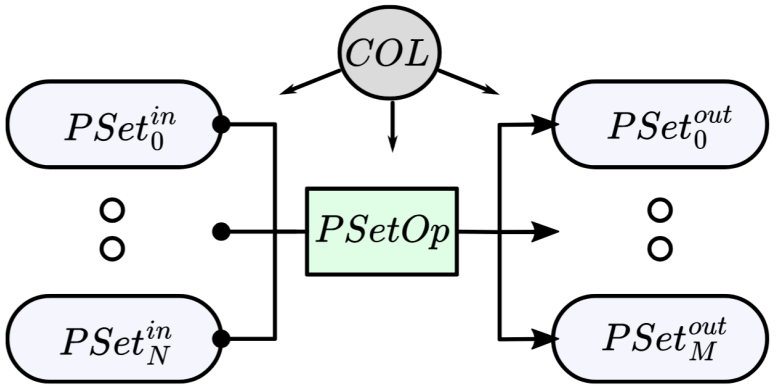
- Cover diverse application types/patterns
- Enable system-wide optimization by RM
- Generic (beyond MPI)



3. Dynamic PSets mechanism

Dynamic PSet Approach:

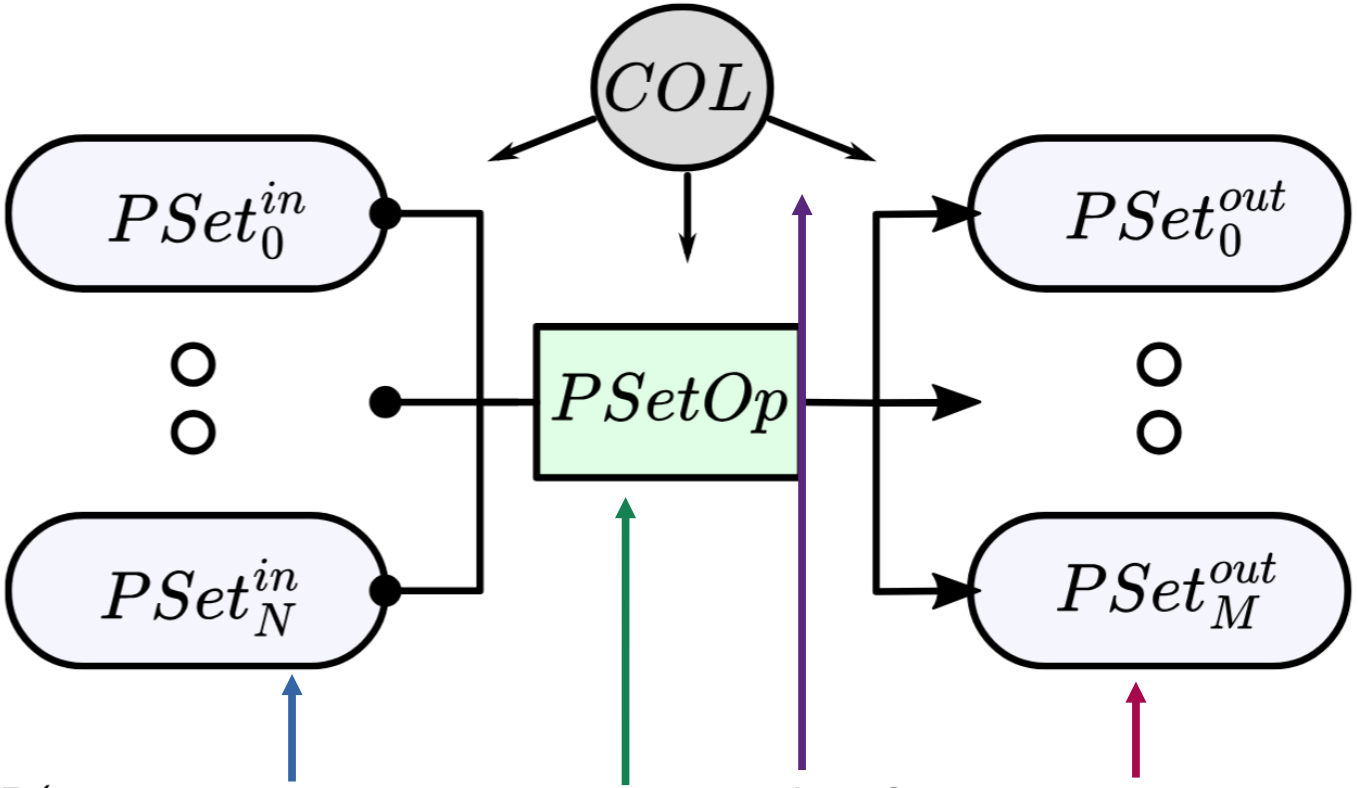
- Graph-based abstraction
  - Vertices = PSets
  - (Hyper)Edges = PSetOps
- Cooperative optimization



Some assumptions

- PSets are immutable
- Dynamically created PSets do not need to appear in the PSet list
- PSets have globally unique name between adjacent software layers → mpi://WORLD is an alias for global uri
- New pset info keys: mpi_dynamic, mpi_alias, (mpi_included, mpi_primary)
- PSet names can be used even if not in list (e.g. group_from_session_pset)
  - The function MPI_GROUP_FROM_SESSION_PSET creates a group newgroup using the provided session handle and process set name. Process set names returned from MPI_SESSION_GET_NTH_PSET for the supplied session are considered valid process set names by MPI. If the pset_name is not considered valid by MPI at the time of the call, MPI_GROUP_NULL will be returned in the newgroup argument.

3. Dynamic PSets mechanism



MPI_SESSION_IPSETOP(session, input_psets, pset_op, col_info, output_psets, output_size, ~~retained,~~ request)

3. Dynamic PSets mechanism

MPI_SESSION_IPSETOP(session, input_psets, pset_op, col_info, output_psets, output_size, retained, request)

IN	session	MPI Session handle (handle)
IN	input_psets	The list of input PSets of the operation (string)
INOUT	pset_op	The PSet Operation (handle)
IN	col_info	The optimization information expressed with the COL (handle)
OUT	output_psets	The list of output PSets of the operation (string)
IN	output_size	Maximum possible size of the output_psets string. '-1' indicates the buffer to be allocated by the MPI. (integer)
IN	re	Flag indicating whether this setop is retained for repeated executions. (logical)
INOUT	request	Request to query status of operation (handle)

3. Dynamic PSets mechanism

- By specifying PSet operations at runtime, applications can express support for resource reconfigurations of specific (sub)sets of processes over time.
- From the resource manager's view, this creates an opportunity space for graph extensions—i.e., reconfigurations—from which it can select options that align with current optimization goals.

We illustrate several PSetOps that reflect common application patterns:

ADD: $P^0_{in} \rightarrow P^0_{out}, P^0_{in} \cap P^0_{out} = \emptyset$

Expresses launch of a task with a new set of processes.

SUB: $P^0_{in} \rightarrow \emptyset$

Expresses termination of a task executed by a set of processes.

SPLIT: $P^0_{in} \rightarrow P^k_{out}, \bigcap_i P^i_{out} = \emptyset \wedge \bigcup_i P^i_{out} = P^0_{in}$

Expresses splitting of a task into subtasks.

UNION: $P^i_{in} \rightarrow P^0_{out}, \bigcup_i P^i_{in} = P^0_{out},$

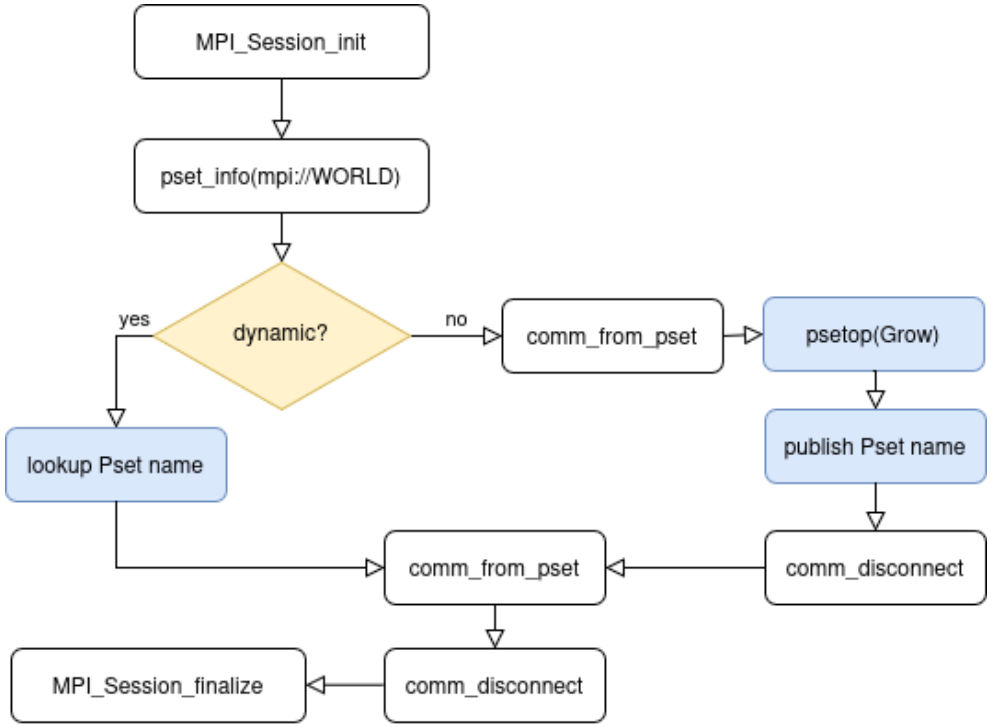
Expresses merging subtasks.

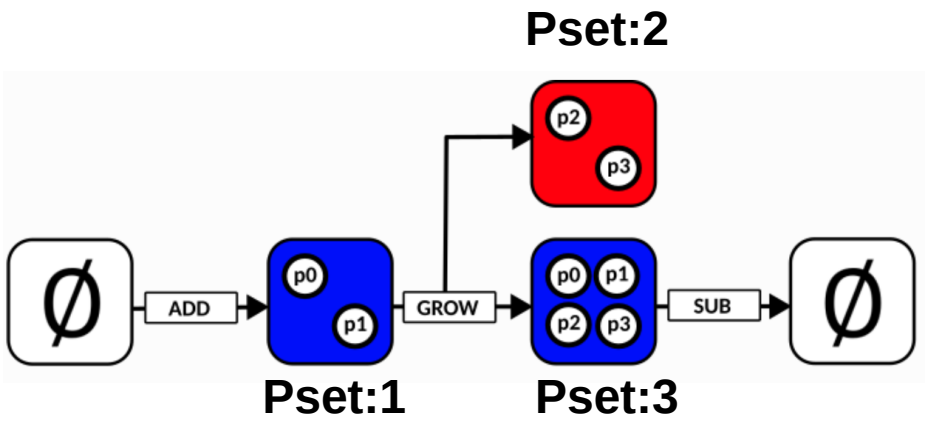
GROINK: $P^0_{in} \rightarrow \{P^0_{out}, P^1_{out}, P^2_{out}\}, P^1_{out} \subset P^0_{out} \wedge P^2_{out} = P^0_{in} \setminus P^0_{out} \cup P^1_{out}.$

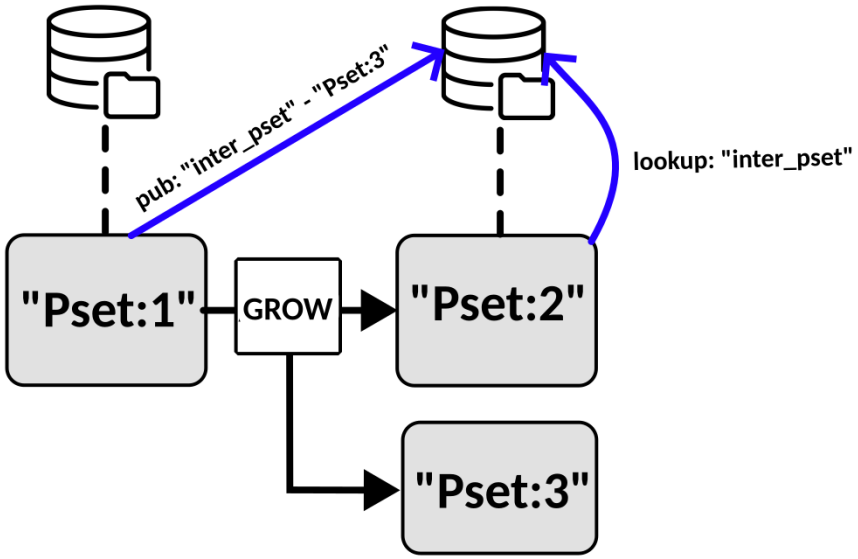
Expresses adding/removing processes to/from a task executed by a PSet.

3. Dynamic PSets mechanism

Example: Grow with publish/lookup



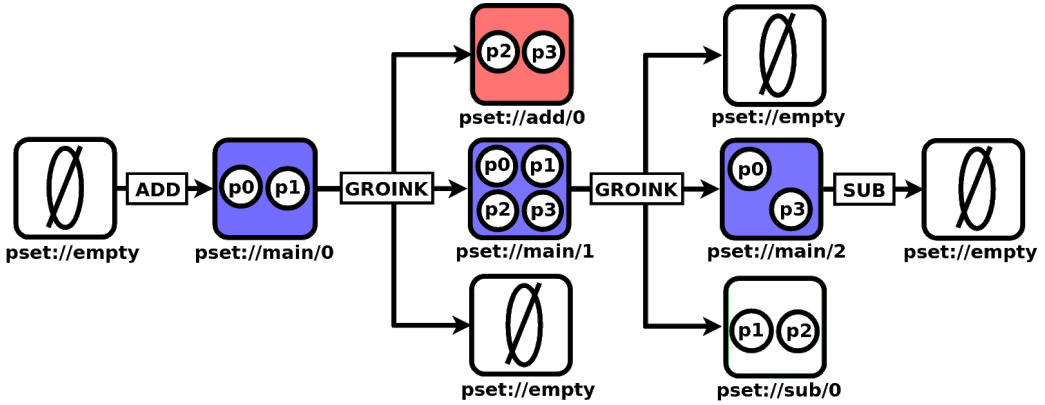




3. Dynamic PSets mechanism

Example: Iterative Application with a single “dynamic” communicator

```
void main() {
    DMR_INIT(user_init_data(), recv_expand());
    DMR_Set_parameter(MIN, MAX, SWEET_SPOT);
    DMR_Inhibit_iter(n_iters);
    for(it = 0; it < ITERATIONS; it++) {
        DMR_RECONFIGURATION( send_expand(),
                             recv_expand(),
                             send_shrink(),
                             recv_shrink());
        compute();
    }
    DMR_FINALIZE(user_free_data());
}
```



4. Use side document mechanism

Motivation: DSL: Cooperative Optimization Language (COL) for dynamic resource optimization

Express:
- Constraints (HW-features, process numbers, ...)
- Performance Models (speedup, energy, accuracy, ...)
- Mapping

→ NOT MPI-specific but needs to be passed to MPI function (MPI_Session_Ipsetop)
→ Decouple expressions: dynamic process structure & optimization information

5. Session Attributes

- Assists with language bindings
- Useful for tools
- Similar functions as for comm, win, datatype:
  - MPI_Session_create_key_val
  - MPI_Session_delete_attr
  - MPI_Session_set_attr
  - MPI_Session_get_attr

Possibly targeting minor release

6. Accessor methods

- getting session handles from other MPI objects (groups, comms, files, windows) to assist tools.
- Functions like
  - MPI_Group_get_session
  - MPI_Comm_get_session
  - …
- MPI_Session_set/get_name?

Possibly targeting minor release

Non-Sessions WG thoughts?

- Reduce number of in person meetings to one per year?
  - Budget constraints
  - Travel restrictions
- WG does not think the pace of MPI standard updates should be reduced.